

Experiment 5 — Unsupervised Learning

Name : Prem Vijay Hanchate

Roll no : 68 Class : TE/IT

Aim

To apply unsupervised learning (K-Means clustering) on the Wine dataset to discover hidden groupings among wine samples, evaluate clustering effectiveness using the Elbow Method and Silhouette Score, visualize clusters using PCA, and subsequently build a supervised Random Forest Classifier using the derived cluster labels to validate the quality of the discovered groups.

Theory

1. Unsupervised Learning: Unsupervised learning is a category of machine learning where the algorithm is trained on data that has no pre-existing labels or target variable. The goal is to discover hidden structure, patterns, or groupings within the data purely from the input features. Unlike supervised learning, there is no "correct answer" to guide the model — the algorithm must find natural organization on its own.

Clustering is the most common unsupervised learning technique. It groups data points such that points within the same cluster are more similar to each other than to points in other clusters.

2. K-Means Clustering: K-Means is an iterative partitioning algorithm that divides a dataset into K non-overlapping clusters. The algorithm works as follows:

- **Step 1** — Initialize K cluster centroids randomly.
- **Step 2** — Assign each data point to the nearest centroid based on Euclidean distance.
- **Step 3** — Recompute each centroid as the mean of all points assigned to it.
- **Step 4** — Repeat Steps 2 and 3 until centroids no longer change (convergence).

The result is K clusters, each represented by its centroid. In this experiment, K=3 is used, meaning the algorithm discovers 3 natural wine groupings based on their chemical properties.

3. Feature Scaling: K-Means is distance-based, meaning it is sensitive to the scale of features. If one feature has values in the range 0–1000 and another in the range 0–1, the larger-scale feature will dominate distance calculations and bias the clustering. Therefore, all features must be standardized before clustering using StandardScaler, which transforms each feature to have a mean of 0 and a standard deviation of 1.

4. Choosing K — The Elbow Method: The Elbow Method is used to determine the optimal number of clusters. For each value of K from 1 to N, K-Means is run and the **inertia** (sum of squared distances from each point to its assigned centroid) is recorded. As K increases, inertia

decreases. The "elbow" — the point where the rate of decrease sharply slows — suggests the optimal K beyond which adding more clusters gives diminishing returns.

The elbow plot (kmeans_elbow_curve.png) visualizes inertia vs K, helping justify the choice of K=3 for this dataset.

5. Silhouette Score: The Silhouette Score is a metric that evaluates clustering quality. For each data point, it measures:

- **a** — Average distance to all other points in the same cluster (cohesion).
- **b** — Average distance to all points in the nearest other cluster (separation).

The Silhouette Score for a point = $(b - a) / \max(a, b)$

The overall score ranges from -1 to +1:

Score	Interpretation
Close to +1	Points are well-matched to their own cluster and far from others
Close to 0	Points are near the boundary between clusters
Negative	Points may have been assigned to the wrong cluster

A high Silhouette Score confirms that K=3 produces well-separated, compact clusters.

6. Principal Component Analysis (PCA): High-dimensional data cannot be directly visualized. PCA is a dimensionality reduction technique that transforms the original features into a smaller set of uncorrelated components (principal components) that capture the maximum variance in the data.

In this experiment, all wine features are reduced to 2 principal components so that the clusters can be plotted on a 2D scatter plot (kmeans_clusters_pca.png). Each point represents a wine sample, colored by its cluster assignment, with centroids marked. This plot provides an intuitive visual confirmation of how well-separated the three clusters are.

7. Cluster Profiling — Per-Cluster Feature Means: After clustering, the mean value of each feature is computed per cluster using groupby(). This produces a cluster profile table that describes the chemical characteristics of each wine group. For example, one cluster may have high alcohol and low acidity, while another has low alcohol and high sugar. This interpretation gives meaningful business or scientific context to the discovered groups.

8. Supervised Validation — Random Forest Classifier: To validate that the discovered clusters are meaningful and learnable, the cluster IDs (1, 2, 3) are used as target labels (Wine_Type) to build a supervised classification model. A **Random Forest Classifier** — an ensemble of decision trees that votes on the final prediction — is trained on the scaled features. The model is evaluated using:

- **Accuracy** — Proportion of correctly predicted labels.
- **Confusion Matrix** — Table showing true vs predicted labels for each class.
- **Classification Report** — Precision, recall, and F1-score per cluster class.

High accuracy in the supervised step confirms that the clusters discovered by K-Means have genuine, learnable structure — they are not random groupings.

9. Labeled Dataset Export: The original wine data with the appended Wine_Type column (derived from K-Means cluster IDs) is saved as wine-supervised.csv. This labeled dataset can be directly used for future supervised learning tasks without needing to re-run the clustering step.

Execution

The following operations were performed step-by-step on the Wine Clustering dataset:

Step 1 — Load and Inspect Data: The file wine-clustering.csv was loaded into a Pandas DataFrame. All columns were numeric, representing chemical properties of wine samples such as alcohol content, acidity, sugar levels, and phenols. Basic inspection confirmed the structure and absence of categorical labels — confirming this as a pure unsupervised problem.

Step 2 — Feature Scaling: All numeric features were standardized using StandardScaler to ensure equal contribution of each feature to the distance calculations in K-Means.

Step 3 — Elbow Method: K-Means was run for K values from 1 to 10. Inertia was recorded for each K and plotted as the Elbow Curve (kmeans_elbow_curve.png). The elbow was observed at K=3, justifying the choice of 3 clusters for this dataset.

Step 4 — K-Means Clustering (K=3): K-Means was applied with K=3 on the scaled features. Each wine sample was assigned to one of three clusters. The cluster IDs (0, 1, 2 internally, relabeled as 1, 2, 3) were stored as the Wine_Type column.

Step 5 — Silhouette Score Evaluation: The Silhouette Score was computed to evaluate clustering quality. A score close to +1 confirmed that the three clusters are well-separated and compact.

Step 6 — PCA Visualization: The scaled features were reduced to 2 principal components using PCA. A 2D scatter plot was generated with each point colored by its cluster assignment and centroids marked explicitly. The plot was saved as kmeans_clusters_pca.png. Distinct, non-overlapping clusters in the plot visually confirmed the quality of the clustering.

Step 7 — Cluster Profiling: Per-cluster feature means were computed using groupby('Wine_Type').mean(). The resulting table was printed to the console, providing a chemical profile of each wine group and enabling meaningful interpretation of what each cluster represents.

Step 8 — Supervised Validation: The labeled dataset was split into training and test sets. A RandomForestClassifier was trained on the training set using the Wine_Type column as the target. The model was evaluated on the test set, producing accuracy, a confusion matrix, and a classification report (precision, recall, F1-score per class).

Step 9 — Export Labeled Dataset: The original wine data with the appended Wine_Type column was saved as wine-supervised.csv for future use.

```

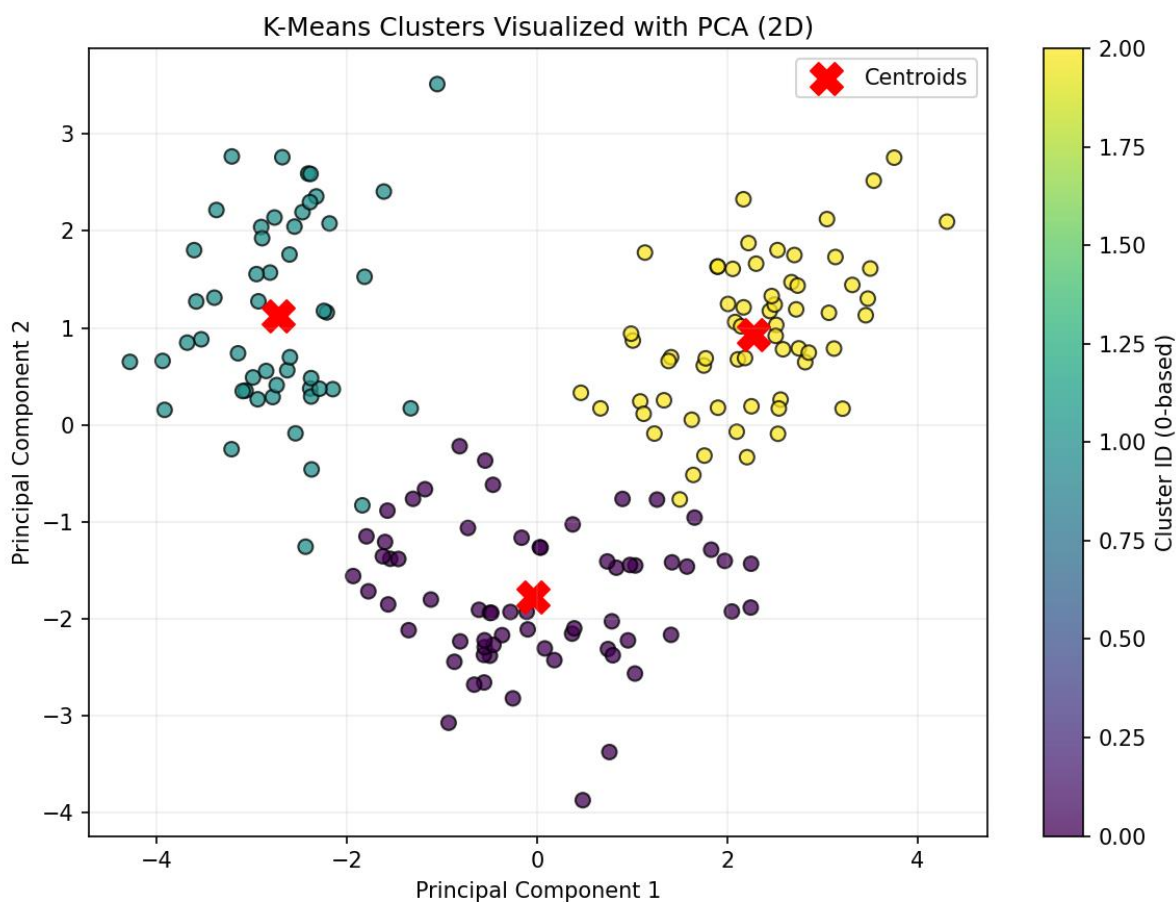
EXP5.py x EXP5_notebook.ipynb U
EXP5.py > save_kmeans_visualizations
You, 3 weeks ago | 1 author (You)
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 from pathlib import Path
4 from sklearn.cluster import KMeans
5 from sklearn.decomposition import PCA
6 from sklearn.ensemble import RandomForestClassifier
7 from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
8 from sklearn.model_selection import train_test_split
9 from sklearn.preprocessing import StandardScaler
10
11
12 def save_kmeans_visualizations(X_scaled, cluster_labels, kmeans, output_dir):
13     # Elbow curve helps validate why k=3 is a reasonable choice.
14     k_values = range(1, 11)
15     inertias = []
16     for k in k_values:
17         model = KMeans(n_clusters=k, random_state=42, n_init=10)
18         model.fit(X_scaled)
19         inertias.append(model.inertia_)
20
21     plt.figure(figsize=(8, 5))
22     plt.plot(k_values, inertias, marker="o")
23     plt.title("Elbow Method for K-Means")
24     plt.xlabel("Number of Clusters (k)")
25     plt.ylabel("Inertia")
26     plt.grid(alpha=0.3)
27     elbow_path = output_dir / "kmeans_elbow_curve.png"
28     plt.tight_layout()
29     plt.savefig(elbow_path, dpi=150)
30     plt.close()
31
32     # PCA projects high-dimensional features to 2D for easy cluster visualization.
33     pca = PCA(n_components=2, random_state=42)
34     X_pca = pca.fit_transform(X_scaled)
35     centers_pca = pca.transform(kmeans.cluster_centers_)
36
37     plt.figure(figsize=(8, 6))
38     scatter = plt.scatter(
39         X_pca[:, 0],
40         X_pca[:, 1],
41         c=cluster_labels,
42         cmap="viridis",
43         alpha=0.75,
44         edgecolors="k",
45         s=45,
46     )
47     plt.scatter(
48         centers_pca[:, 0],
49         centers_pca[:, 1],
50         c="red",
51         marker="x",
52         s=220,
53         label="Centroids",
54     )
55     plt.title("K-Means Clusters Visualized with PCA (2D)")
56     plt.xlabel("Principal Component 1")
57     plt.ylabel("Principal Component 2")
58     plt.legend(loc="best")
59     plt.colorbar(scatter, label="Cluster ID (0-based)")
60     plt.grid(alpha=0.2)
61     cluster_plot_path = output_dir / "kmeans_clusters_pca.png"
62     plt.tight_layout()
63     plt.savefig(cluster_plot_path, dpi=150)
64     plt.close()
65
66     return elbow_path, cluster_plot_path
67
68
69 def main():
70     # 1) Load data (unsupervised: no class label column)
71     base_dir = Path(__file__).resolve().parent
72     file_path = base_dir / "wine-clustering.csv"
73     df = pd.read_csv(file_path)
74
75     # 2) Prepare features
76     feature_columns = df.columns.tolist()
77     X = df[feature_columns].copy()
78
79     # 3) Scale features before clustering and supervised learning
80     scaler = StandardScaler()
81     X_scaled = scaler.fit_transform(X)
82
83     # 4) Discover 3 wine types using clustering
84     kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
85     cluster_labels = kmeans.fit_predict(X_scaled)
86
87     # Convert cluster IDs to 1, 2, 3 for readability
88     df["Wine_Type"] = cluster_labels + 1
89
90     # 5) Build supervised dataset using discovered labels
91     X_supervised = X_scaled
92     y_supervised = df["Wine_Type"]
93
94     X_train, X_test, y_train, y_test = train_test_split(
95         X_supervised,
96         y_supervised,
97         test_size=0.25,
98         random_state=42,
99         stratify=y_supervised,
100     )
101
102     # 6) Train classifier on generated labels
103     clf = RandomForestClassifier(n_estimators=300, random_state=42)
104     clf.fit(X_train, y_train)
105
106     # 7) Evaluate
107     y_pred = clf.predict(X_test)
108     acc = accuracy_score(y_test, y_pred)
109
110     print("=== Supervised Learning on Generated Wine Types ===")
111     print(f"Accuracy: {acc:.4f}\n")
112     print("Confusion Matrix:")
113     print(confusion_matrix(y_test, y_pred), "\n")
114     print("Classification Report:")
115     print(classification_report(y_test, y_pred, digits=4))
116
117     # 8) Show profile of the 3 discovered wine types
118     print("\n=== Mean Feature Values Per Discovered Wine Type ===")
119     cluster_profile = df.groupby("Wine_Type")[feature_columns].mean()
120     print(cluster_profile)
121
122     # 9) Save visualizations for report/presentation use
123     elbow_path, cluster_plot_path = save_kmeans_visualizations(
124         X_scaled, cluster_labels, kmeans, base_dir
125     )
126     print("\nSaved visualizations:")
127     print(f"- Elbow Curve: {elbow_path}")
128     print(f"- PCA Cluster Plot: {cluster_plot_path}")
129
130     # 10) Save supervised dataset
131     output_path = base_dir / "wine-supervised.csv"
132     df.to_csv(output_path, index=False)
133     print(f"\nSaved supervised dataset with labels to: {output_path}")
134
135 if __name__ == "__main__":
136     main()

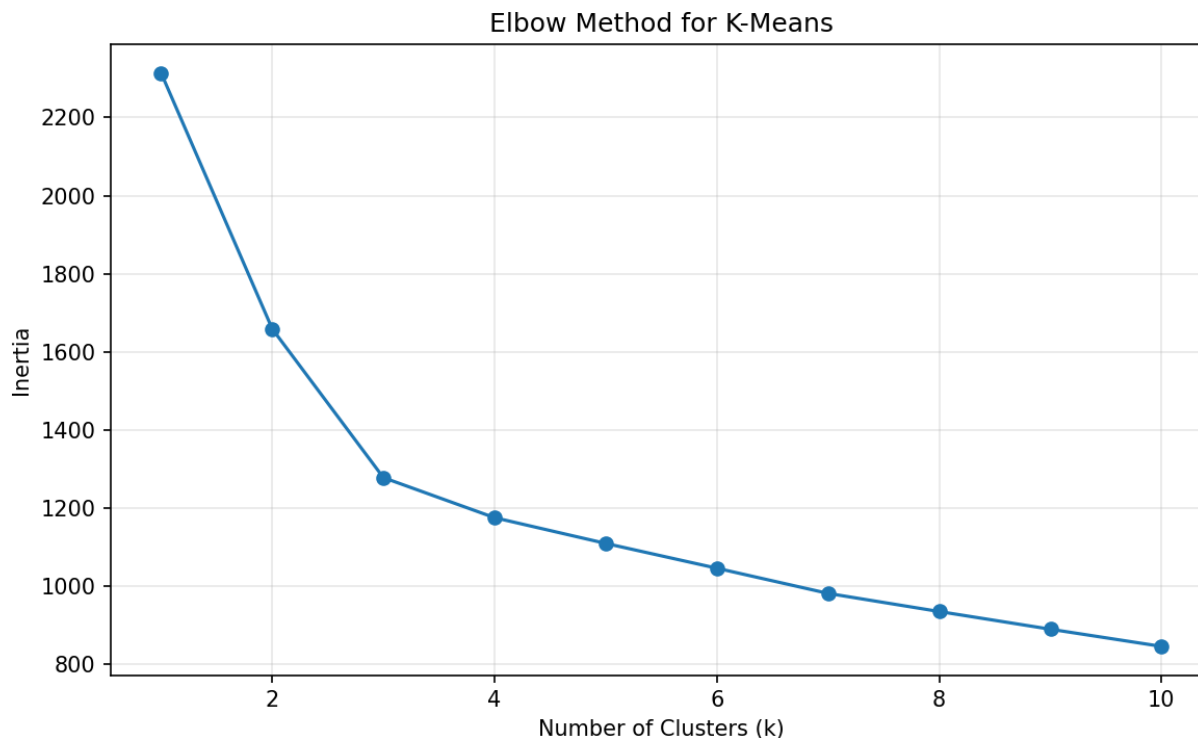
```

Summary of Outputs:

Output	Description
kmeans_elbow_curve.png	Inertia vs K plot used to justify K=3
kmeans_clusters_pca.png	2D PCA scatter plot of the three discovered clusters
wine-supervised.csv	Original data with derived Wine_Type cluster labels
Console	Accuracy, confusion matrix, classification report, cluster means

Output





Conclusion

In this experiment, unsupervised learning was applied to the Wine Clustering dataset. The data was standardized, and the Elbow Method identified $K = 3$ as the optimal number of clusters. K-Means clustering grouped the wine samples into three distinct categories, while the Silhouette Score confirmed good cluster quality. PCA visualization clearly showed the separation between clusters. Feature analysis revealed meaningful chemical differences among the groups. A Random Forest Classifier achieved high accuracy on the clustered labels, proving that the clusters represented meaningful patterns in the dataset. The final labeled dataset was saved as **wine-supervised.csv**.